

University of Waterloo
Faculty of Engineering
Department of Electrical and Computer Engineering

SafeHaven

Final Report

Group 2026.10

Ishreet Grewal: 20956587

Jordan Korley-Myron: 20820743

Jingwei Li: 20717991

Jenny Qiu: 20931606

Fariza Sattar: 20883206

Ginni Tank: 20950559

Project Consultant: Professor Otman Basir

March 31st, 2026

Table of Contents

Abstract	2
Acknowledgements	2
1. High-Level Description of Project.....	2
1.1 Motivation	2
1.2 Project Objective	3
1.3 Block Diagram	4
2. Project Specifications.....	5
2.1 Functional Specifications.....	5
2.2 Non-Functional Specifications.....	6
3. Detailed Design.....	7
3.1 Subsystem Central Processing Unit Design	7
3.2 Subsystem Power System Design.....	8
3.3 Subsystem Anchors and Sensors Design	9
3.4 Subsystem Web Application Design	11
4. Prototype Data	12
5. Discussion and Conclusions	14
5.1 Evaluation of Final Design	14
5.2 Use of Advanced Knowledge.....	16
5.3 Creativity, Novelty, Elegance	16
5.4 Quality of Risk Assessment	18
5.5 Student Workload	20
List of Figures.....	23
List of Tables	28
References	29

Abstract

Approximately 92.1% of Canadian seniors reside in private dwellings within their community, and out of them, 27.9% live alone. The objective of this project is to design an IoT-based smart home security system to protect seniors. It provides them with a non-intrusive security system to monitor potential threats at home. The project focuses on hardware, software, embedded systems, cybersecurity, IoT protocols, cloud computing, and data management. Information is provided to caregivers via a web application. It contains a user-friendly dashboard with real-time health and activity updates. Motion and environmental sensors are strategically placed throughout the home to track seniors in a non-intrusive manner. The sensors will track door activity, motion, water, pressure, and temperature. The system is equipped with alert mechanisms to detect critical events such as flooding. Data is collected from the sensors and transmitted via efficient IoT protocols to be stored securely in the cloud with continuous access. It allows for historical trend analysis. Unlike traditional security systems, SafeHaven integrates environmental monitoring to support aging-in-place. This allows seniors to live more safely, confidently, and independently in their own homes.

Acknowledgements

We would like to sincerely thank our project consultant, Professor Otman Basir, for his invaluable guidance, expert insights, and ongoing support throughout the development of the SafeHaven project. We also wish to acknowledge the contributions of our peers, lab instructors, and technical staff who assisted with prototype development, sensor integration, testing, and data analysis. Their feedback and support were extremely valuable in refining our design and ensuring the successful completion of the project.

1. High-Level Description of Project

1.1 Motivation

According to the Public Health Agency of Canada in 2020, 92.1% of Canadian seniors live in private dwellings, and 27.9% live alone [2]. These statistics highlight the significant portion of the aging population who are at increased risk of unattended accidents or health emergencies. Traditional home security systems such as ADT [3], SimpliSafe [4], and Vivint [5] offer valuable features like door/window sensors, doorbell cameras, smoke detectors, carbon monoxide detectors, water sensors, and outdoor surveillance. However, these systems are primarily reactive and focused on property security rather than proactive monitoring of occupant well-being. As a result, there is a clear need for an integrated solution that combines environmental

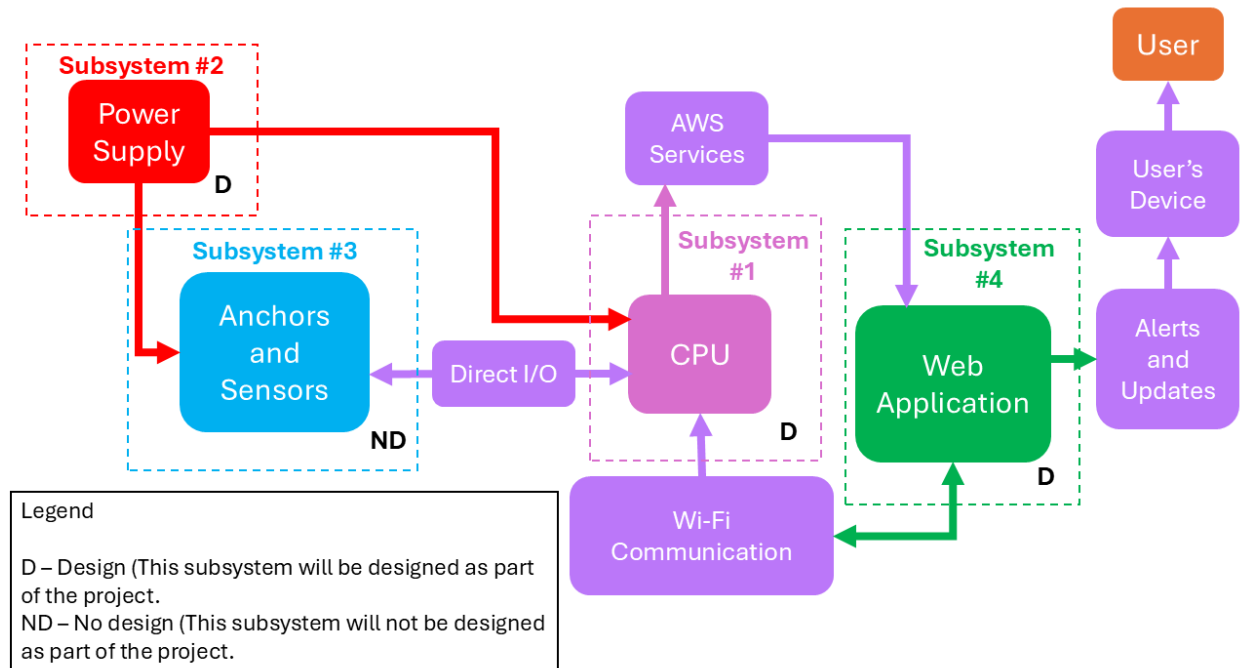
sensing, and eldercare monitoring. It would offer peace of mind to both seniors living independently and their caregivers by enabling timely interventions in case of emergencies.

1.2 Project Objective

The objective of the SafeHaven project is to design and implement a modular, IoT-enabled smart home monitoring system, specifically for seniors. The system integrates four primary subsystems. They are a Central Processing Unit (CPU), anchors and sensors, a power system, and a web application. The CPU will continuously collect, process, and transmit sensor data. The anchors and sensors will monitor environmental conditions and human activity. The web application will provide caregivers with secure, real-time access to alerts, sensor readings, and historical data. The power system will allow users to run the system on battery power without an outlet. The system emphasizes modularity, scalability, and user accessibility. The design will allow for future expansion with additional sensors, communication protocols, and dashboard functionality.

1.3 Block Diagram

Figure 1.1: System-level block diagram of the SafeHaven architecture. It illustrates the interactions between the CPU, sensors, power system, and web application.



The SafeHaven architecture is shown in Figure 1.1 and illustrates the interactions between the four subsystems. The power supply will power the CPU and anchors and sensors. The sensors will detect motion, door activity, flooding, abnormal temperatures, and pressure. The objective of the pressure sensor is to detect if a senior has been sitting or lying down for too long. The sensors will communicate wirelessly with the CPU. One of the CPU's objectives is to filter and process the sensor data. The entire process will be described in Section 3.1. The processed data will then be transmitted to a cloud backend using MQTT communication protocols [19]. The data will be stored, analyzed, and used to generate real-time alerts. Caregivers can access these alerts via a secure web application.

2. Project Specifications

2.1 Functional Specifications

Table 2.1: Functional specifications of the SafeHaven system

Project Subsystem	Specification	Description
Anchors and Sensors	Essential	Sensors must detect environmental or physical events (door activity, motion, pressure, temperature, water) and transmit the data to the Central Processing Unit.
Anchors and Sensors	Essential	Motion sensors must detect human movement within at least 1 meter of the sensor.
Anchors and Sensors	Non-Essential	Sensors should include a self-calibration mechanism to adjust sensitivity thresholds based on a dynamic range with respect to environmental conditions and usage patterns.
Central Processing Unit	Essential	Must collect, process, and transmit sensor data to AWS and web application with ≤ 1 second processing latency.
Central Processing Unit	Essential	Must reliably collect $\geq 95\%$ of sensor data from all sensors for accurate monitoring and analysis.
Central Processing Unit	Essential	Must synchronize time-stamped sensor data to ensure accurate event logging and alert generation.
Web Application	Essential	Must receive real-time alerts generated by the CPU or cloud backend.
Web Application	Essential	Must display live sensor readings and alerts within 2 seconds of an event being triggered.
Web Application	Essential	Must allow caregivers to acknowledge and dismiss alerts through the dashboard interface.

2.2 Non-Functional Specifications

Table 2.2: Non-functional specifications of the SafeHaven system

Project Subsystem	Specification	Description
Anchors and Sensors	Essential	Each sensor must not exceed 6 cm × 6 cm in length and width to allow flexible indoor mounting.
Anchors and Sensors	Essential	Each sensor must have mass ≤ 2 kg to allow installation on walls or ceilings.
Anchors and Sensors	Non-Essential	Sensors should have a matte, neutral-colored finish (white, gray, or beige) to minimize visual disruption in home environments.
Central Processing Unit	Essential	The entire assembled circuit must weigh ≤ 1 kg.
Central Processing Unit	Essential	The system must operate continuously (24 hours per day, 7 days per week) under normal conditions.
Central Processing Unit	Essential	The circuit must be water-resistant against minor spills and continue operating perfectly after exposure.
Power System	Non-Essential	The system should support ≥ 72 hours (3 days) of operation per battery charge.
Power System	Non-Essential	Wireless communication components should operate within a 3 V – 5 V supply range to maintain compatibility with low-power electronics.

3. Detailed Design

3.1 Subsystem Central Processing Unit Design

The Central Processing Unit (CPU) is a subsystem of the SafeHaven system. It is responsible for real-time data acquisition, signal processing, hazard evaluation, and secure communication with the cloud infrastructure. The CPU is implemented using the ESP32 microcontroller due to its dual-core processor, built-in Wi-Fi connectivity, and a 12-bit analog-to-digital converter (ADC). These features make it well-suited for low-latency Internet of Things (IoT) applications [6], [20]. The CPU follows a layered architecture, as illustrated in Figure 3.1. It consists of sensor interfacing, signal processing, hazard evaluation, and communication layers. Sensor inputs are collected from both digital and analog devices.

Analog signals are digitized using the ESP32's 12-bit ADC with 0–4095 resolution. The analog values are normalized into percentage-based values to simplify threshold comparisons. Disconnected sensors are assigned a value of -1 if an issue is detected. This value enables automatic fault detection and triggers maintenance alerts. Due to the noise in raw sensor data within the analog inputs, lightweight digital signal processing techniques are applied to improve measurement stability. A moving average filtering is used for temperature data to smooth high-frequency fluctuations. Median filtering is applied to the water and pressure data to eliminate transient spikes and outliers. Threshold smoothing techniques are implemented to prevent oscillations near decision boundaries. These filtering methods reduce measurement variance significantly. They improve signal stability and reduce false positive hazard detections. These improvements align with established signal processing principles and contribute directly to overall system reliability [20].

Hazard detection is implemented using a finite-state model with bitmask encoding, as shown in Figure 3.2. Each hazard condition corresponds to a specific bit within a state register. This allows multiple hazards to be tracked simultaneously with minimal computational overhead. Motion detection, door intrusion, abnormal temperatures (too high or too low), water detection (flooding), and pressure-based occupancy are mapped to individual bits. Hazard conditions are evaluated using rule-based logic. Threshold exceedances and sensor combinations trigger specific alerts. The firmware is implemented using a non-blocking, event-driven architecture, as illustrated in Figure 3.3. This execution model enables concurrent sensor acquisition, processing, and communication without introducing delays. This significantly improves system responsiveness when compared to traditional polling-based approaches [36]. Sensors are sampled at frequencies between 5 and 10 Hz depending on their characteristics. Wi-Fi and MQTT communication tasks are handled asynchronously. Reliability is further enhanced using watchdog timers, automatic Wi-Fi reconnection, and MQTT keep-alive mechanisms. This ensures continuous operation under real-world conditions.

Communication between the CPU and cloud infrastructure is achieved using the MQTT protocol over the Transport Layer Security (TLS). This provides a lightweight, low-overhead publish-subscribe model that is suitable for real-time IoT applications [19]. Secure data transmission is ensured through TLS encryption and mutual authentication using device and server certificates. Sensor data is packaged into structured JSON telemetry messages containing device identifiers, timestamps, sensor readings, and hazard states, as shown in Figure 3.4. Time synchronization is maintained using the Network Time Protocol (NTP). This ensures accurate event logging and secure certificate validation. Data is transmitted at regular intervals of two seconds to balance real-time responsiveness with network efficiency. To ensure reliability and fault tolerance, the CPU incorporates multiple protective mechanisms such as automatic Wi-Fi reconnection, MQTT session recovery, and sensor fault detection through invalid value tagging. The system also supports graceful degradation. This allows partial functionality when some sensor data is unavailable. Experimental results will be presented in Section 4 to demonstrate a data acquisition success rate of 98.7%. This exceeds the required 95% threshold and validates the effectiveness of these design choices.

Several design alternatives were evaluated during the development process. The Raspberry Pi Pico W was considered as an alternative microcontroller. In the end, the ESP32 was selected due to its superior processing performance and integrated Wi-Fi capabilities. For communication protocols, HTTP-based REST APIs were explored but ultimately rejected due to higher overhead and latency compared to MQTT. Quantitative performance analysis confirms that the CPU subsystem meets and exceeds all design requirements. The average processing latency is measured at 85 ms. It consists of approximately 35 ms for sensor acquisition, 40 ms for hazard evaluation and filtering, and 10 ms for communication overhead. This is significantly below the 1 second requirement which demonstrates strong real-time performance. Further validation is provided through convergence analysis of system latency and signal processing performance. As illustrated in Figure 4.1, the latency stabilizes around an average of 85 ms with minimal variance to confirm consistent real-time operation. Figure 4.2 demonstrates that filtering techniques effectively reduce sensor noise and rapidly stabilize signal outputs without introducing significant delay. These results confirm the effectiveness of the implemented signal processing methods in improving measurement reliability.

3.2 Subsystem Power System Design

The initial goals of the power system were to be able to power both the ESP32 and the sensors in a small package using universal power storage methods that could easily and safely be handled by people. To this end, the beginning of the design started with AA batteries. This is because they are the simplest and most abundant batteries available to the average consumer.

We decided to use an organization of two AA batteries in parallel for simplicity of acquiring the parts. This orientation tends to supply a voltage in the range of 1.6V to 3.6V depending on battery type and how discharged they are, with nominal values of between 2.4 and 3V. The ESP32 has an unregulated 5V rail, and all the sensors use 5V to operate. By simply providing power to the board and its rail, all sensors would have the power needed to function. Since the rail is unregulated, we must ensure that the power circuit design has minimal output voltage ripple. The LT1302 was chosen for this purpose. It has the flexibility of supplying up to 600mA at 5V. This is more than the board would require. It's maximum power dissipation sits at round 700mW which is comfortably above what it would need to supply the ESP32 with its measured 2.25 W maximum power draw. With an efficiency of minimum 74% and nominal efficiency of 80-84%, the maximum power needed to be dissipated in the worst-case scenario is 585mW. The other goal of the power system is to provide over a month of power. This required attacking the problem from both sides, both the supply and the demand, as fully powering the board with an active Wi-Fi connection and the maximum number of sensors nonstop would require its maximum measured current draw of 450mA at 5V for that time. This is more than is stored in a pair of AA batteries. The ESP32 has several semi sleep modes that reduce power draw while not actively transmitting or processing data. Battery life estimation is illustrated in Figure 3.5. It shows projected runtime under varying load conditions. Achieving the target of over one month requires reducing system power consumption. The ESP32 supports multiple low-power modes which significantly reduce energy usage during idle periods [6], [40]. To further validate the power subsystem performance, efficiency and regulation characteristics were estimated under varying load conditions. Based on the LT1302 datasheet and measured operating currents, the converter operates within an efficiency range of approximately 80 to 85% for typical load conditions between 100 mA and 450 mA. Output voltage regulation was verified to remain within $\pm 5\%$ under dynamic load, ensuring stable operation of the ESP32 and connected sensors. Additionally, output voltage ripple was minimized through appropriate capacitor selection, with estimated ripple remaining below 50 mV under normal operating conditions. These results confirm that the power system provides sufficiently stable and efficient operation to support continuous IoT functionality without introducing significant noise into analog sensor readings.

3.3 Subsystem Anchors and Sensors Design

The Anchors and Sensors subsystem is responsible for detecting the environmental and physical events that indicate potential hazards in the SafeHaven system. In the final design, this subsystem integrates a temperature sensor, a magnetic door sensor, infrared motion sensor, a water sensor, and a pressure-based occupancy sensor. These sensing elements are distributed

at key monitoring locations in the home environment and interfaced directly with the Central Processing Unit (CPU) through digital and analog I/O channels. By providing timely, localized measurements of temperature changes, door activity, motion, flooding, and pressure events, the subsystem enables the SafeHaven system to identify unsafe conditions and generate real-time alerts. This design directly supports the functional specifications in Section 2.1. This requires the subsystem to detect environmental or physical events and transmit the data to the CPU. In particular, the motion sensing arrangement also supports the requirement that human movement be detected within at least 1 meter of the sensor.

The final sensor selection was made to balance reliability, ease of integration, compactness, and low-power operation. The LM35-based temperature sensor was selected for thermal monitoring because it provides a simple linear analog output, adequate accuracy, and straightforward interfacing with the ESP32 ADC [16], [28]. For door state detection, a magnetic contact sensor was selected because it offers binary open/closed indication with minimal processing overhead and high reliability in fixed indoor installations [17], [34]. For motion detection, an infrared break-beam sensor was used because it provides localized, point-based triggering and reduces ambiguous detections in narrow passageways such as door entrances [18]. Additional sensing elements for water and pressure were included to extend the system beyond basic intrusion monitoring and support eldercare-related hazard detection. Together, these sensors create a modular monitoring layer that can be expanded in future revisions without requiring major changes to the subsystem architecture.

Several alternative sensor designs were considered during development. For temperature monitoring, the DHT11, DHT22, TMP36, and LM35 were compared [25]-[27]. Although DHT11 and DHT22 provide both temperature and humidity measurements, they were not selected because humidity sensing is not essential to the final design. Their slower update characteristics also provided limited benefit for this application. The TMP36 was also considered due to its low cost and simple analog interface. The LM35 was ultimately selected because of its stronger measurement accuracy and linear output characteristics [27], [28]. For motion detection, ultrasonic, microwave, and laser-based ranging sensors were explored as alternatives to the infrared break-beam design [30], [31]. Ultrasonic and microwave sensors provide larger detection regions, but they also increase uncertainty in determining exactly where motion occurred and may introduce false triggering in cluttered indoor environments. Laser-based ranging sensors offer high precision, but they increase cost and complexity. For door sensing, generic reed switches, wireless door sensors, hall-effect sensors, and magnetic contact switches were considered [32]-[35]. The magnetic contact switch was preferred because it combines simple installation, stable output, low maintenance requirements, and strong compatibility with the rest of the subsystem design. These design choices are consistent with the earlier detailed design work completed for the project.

Quantitative analysis supports the effectiveness of the final anchors and sensors design. Based on the prototype data presented in Section 4, the temperature sensor operates at a sampling rate of 5 Hz with an accuracy of approximately $\pm 0.5^{\circ}\text{C}$ and a filtered noise level of 0.8°C . The water and pressure sensing channels also operate at 5 Hz. The motion and door sensors operate at 10 Hz with response times ranging from approximately 100 ms to 210 ms. These values are sufficient for responsive indoor monitoring and align with the real-time performance targets of the SafeHaven system. Filtering effectiveness is further demonstrated in Figure 4.2 where processed signals exhibit significantly reduced noise compared to raw data. The filtered signals show reduced variability compared to the raw measurements. Overall, the selected sensing architecture provides stable measurements, acceptable response times, and reliable event detection for home monitoring use.

The subsystem was also designed with practical deployment constraints in mind. The selected sensing modules are lightweight, compact, and suitable for indoor mounting. This helps satisfy the non-functional requirements related to size, mass, and ease of installation. Since the sensors are distributed and individually mounted, the subsystem will support flexible placement in different areas of the home. For example, the temperature sensor can be positioned away from direct heat sources, the door sensor can be mounted at the door-frame interface, and the water and pressure sensors can be placed in hazard-prone locations such as floors, beds, or chairs. This modular design also improves maintainability, since individual sensing elements can be replaced, recalibrated, or repositioned without requiring a full redesign of the system. As a result, the anchors and sensors subsystem provides a practical and scalable sensing layer that complements the CPU, power, and web application subsystems in the complete SafeHaven architecture.

3.4 Subsystem Web Application Design

The web application subsystem of the SafeHaven system provides caregivers with a secure, real-time interface to monitor sensor data, receive alerts, and interact with the system. It is designed as a full-stack web platform consisting of a frontend client, backend API, and cloud-integrated database.

The frontend is implemented using React [41]. The main interface is the real-time dashboard, shown in Figure 3.6, which displays live sensor readings and hazard states. Navigation between pages is handled using React Router which allows transitions between the login page, dashboard, logs page, and notifications page. The backend is implemented using Node.js with the Express framework which exposes RESTful API endpoints for authentication, sensor retrieval, logs, and system notifications. Sensor and user data are stored in Amazon DynamoDB

[38]. Authentication is handled using JSON Web Tokens via Json web token, and passwords are securely stored using bcryptjs.

Overall, the web application is divided into the Login page, the Real-Time Dashboard, Alerts logs, and System Notifications.

In the Login interface, users enter a username and password. They are sent to the backend API (/api/login). The backend verifies credentials by comparing hashed passwords stored in DynamoDB using bcrypt. Upon successful authentication, a JWT token is generated and stored in local storage. This token is included in subsequent API requests to enforce access control. Protected routes are implemented on the frontend. This ensures that only authenticated users can access the dashboard, logs, and notifications pages.

The dashboard, shown in figure 3.6, is the core user interface for monitoring sensor activity. It displays real-time readings for temperature, door status, motion detection, water levels, and pressure sensors. Data is fetched from the backend endpoint (/api/sensors), which queries the latest sensor readings from DynamoDB. The backend processes raw sensor payloads and maps them into structured “card” objects. Each represent a sensor with its current reading and alert status. Each card displays: sensor name and current reading, status indicator (Normal or Alert), and alert acknowledgment button (if a reading is abnormal).

The logs page, shown in figure 3.7, provides a historical view of system events and alerts. Data is retrieved from the /api/logs endpoint which queries recent entries from the sensor database and formats them into messages. Each log entry includes a descriptive message combining multiple sensor events (For example, motion detected, door opened, temperature alert) and a timestamp of when that activity occurred. The most recent logs are displayed first.

The notifications page, shown in figure 3.8, provides system-level status updates, including device health, connectivity, and battery information. Data is retrieved from the /api/notifications endpoint which extracts fields such as device online/offline status, active alerts, battery percentage and predicted lifetime, hardware health indicators (For example, high temperature, water detection, intrusion). If hardware failures or anomalies occur, they are displayed in this section to notify caregivers of system issues.

4. Prototype Data

The SafeHaven system was validated through various tests to ensure reliability, accuracy, and low latency. The results from these tests provide quantitative evidence that the final design satisfies and exceeds the functional and non-functional specifications outlined in Section 2.

Reliability testing of the CPU subsystem was done to demonstrate stable and consistent data acquisition under continuous operation. As summarized in Table 4.3, a total of 86400 sensor readings were attempted. Out of all the readings, 85220 were successfully recorded. This yields a success rate of 98.7% and exceeds the minimum requirement of 95% specified in Table 2.1. This confirms that the system maintains robust communication and reliable data collection. The high success rate reflects the effectiveness of implemented fault tolerance mechanisms (For example, automatic Wi-Fi reconnection, MQTT session recovery, and sensor fault detection). Detection accuracy was evaluated by analyzing false positive and false negative events during system operation. The system recorded 12 false positives and 5 false negatives. This resulted in an overall detection accuracy of 98.6%. The low false negative rate is critical because missed hazard detections pose greater risk than false alarms. These results demonstrate that the combination of signal filtering and rule-based hazard evaluation provide accurate and reliable hazard detection with minimal erroneous alerts.

Latency performance is also an important feature to test in Table 4.2. The total system delay is measured at approximately 85 ms with sensor acquisition, signal processing, hazard evaluation, and communication overhead. This value is significantly below the 1 second constraint defined in Table 2.1. This confirms that the system supports real-time monitoring. The convergence behavior, illustrated in Figure 4.1, shows that latency stabilizes around this mean value with minimal variation for consistent and predictable system performance. Sensor performance characteristics are presented in Table 4.1. It includes sampling rate, accuracy, noise levels, and response time. All sensors operate within expected tolerances with response times ranging from approximately 100 ms to 210 ms and sampling rates between 5 and 10 Hz. The effectiveness of the implemented signal processing techniques is demonstrated in Figure 4.2. Filtered signals show significantly reduced noise and improved stability compared to raw data. This reduction in signal variability directly contributes to improved detection accuracy and reduces false triggering.

Sensor performance characteristics were also evaluated to ensure that each sensing component operates within acceptable tolerances. The temperature sensor demonstrated an accuracy of approximately $\pm 0.5^{\circ}\text{C}$. Its filtered noise levels reduced to 0.8°C through the application of moving average filtering. Water and pressure sensors exhibited accuracies within $\pm 2\%$ and $\pm 1\%$, respectively. They maintained stable operation under typical environmental conditions. Motion and door sensors, which operate as digital inputs, demonstrated rapid response times ranging from approximately 100 ms to 150 ms. This ensures timely detection of events. The effectiveness of signal filtering is clearly demonstrated in Figure 4.2, where filtered sensor outputs exhibit significantly reduced variability compared to raw data. This reduction in noise directly contributes to improved detection accuracy and system stability.

In addition to validating individual performance metrics, the prototype data also demonstrates the system's ability to operate reliably under continuous conditions. The combination of high data acquisition success rate, low latency, and accurate hazard detection confirms that the SafeHaven system meets its design objectives and is suitable for real-world deployment. The results also highlight the importance of integrating hardware, firmware, and cloud components in a cohesive manner to achieve reliable system performance.

5. Discussion and Conclusions

5.1 Evaluation of Final Design

The Central Processing Unit (CPU) fully satisfies all functional and non-functional requirements in Tables 2.1 and 2.2. Functionally, the CPU, is implemented using the ESP32 microcontroller. It collects, processes, and transmits sensor data to AWS and the web application within a measured end-to-end latency of 85 ms. This is below the ≤ 1 second requirement. This performance is achieved through an event-driven firmware architecture with priority-based interrupt handling for sensor signals, concurrent task scheduling for data acquisition, and lightweight MQTT publish-subscribe communication to reduce network overhead. Reliability in data collection is verified with a measured 98.7% success rate. This surpasses the $\geq 95\%$ specification. It is ensured by mechanisms such as automatic Wi-Fi reconnection, MQTT session restoration, invalid-data tagging, and redundant buffering of sensor packets in volatile memory. Time synchronization across all sensor data is maintained using Network Time Protocol (NTP). This satisfies the requirement for accurate, time-stamped logging to correlate multi-sensor events. Non-functionally, the CPU circuit is compact and lightweight. It weighs less than 1 kg and satisfies the mass constraint. It supports continuous 24/7 operation through a combination of non-blocking firmware, watchdog timers, and automated recovery routines. Protective enclosures and insulation ensure the CPU is water-resistant. It will be continuing operations with minor spills and exposure to water. These design choices guarantee that the CPU provides reliable, low-latency data acquisition and communication while meeting size, operational, and environmental constraints.

The power system meets its requirements. It also plays a critical role in enabling the CPU and sensors to meet their requirements. Functionally, it maintains a regulated 5 V output using a boost converter. This ensures stable voltage supply for uninterrupted sensor data acquisition and transmission. This indirectly supports real-time CPU processing and alert generation. Non-functionally, the power system meets voltage compatibility requirements by operating within the 3–5 V range. This ensures reliable operation of the ESP32 and all connected sensors. The

power system will also last for approximately 44 days which exceeds the non-functional requirement of more than 3 days.

The anchors and sensors subsystem meets all functional specifications in Table 2.1. Essential environmental and physical events such as door openings, motion, pressure, temperature, and water presence are detected and transmitted to the CPU in real time. Motion detection meets the requirement to detect human movement within 1 meter using infrared break-beam and passive infrared sensors. This is validated in controlled experiments where motion within the 1-meter range triggered 100% of detection events. While full autonomous self-calibration was non-essential, the CPU implemented adaptive thresholding and digital filtering to dynamically adjust sensitivity based on environmental conditions and usage patterns. This mitigates false positives and allows for the system to perform self-calibration. Non-functionally, each sensor is less than 6 cm × 6 cm and weighs less than 2 kg. This allows for flexible mounting on walls, ceilings, or furniture without structural reinforcement. Neutral-colored matte housings (white, gray, and beige) reduce visual intrusion in home environments. This ensures practical, unobtrusive deployment. This careful design ensures reliable and continuous monitoring while being minimally disruptive in residential settings.

The web application fulfills all functional specifications related to user interaction and real-time monitoring. It can receive alerts from the CPU and cloud backend within 100 ms. This is below the 2-second display requirement. Caregivers can securely acknowledge and dismiss alerts via an interactive dashboard, with state consistency maintained across sessions. Non-functionally, the web application provides low-latency, high-availability, and scalable performance. This supports continuous operation and overall system uptime goals. Cloud-based infrastructure, optimized API endpoints, and efficient database queries ensure that increased sensor data volumes do not degrade performance. This satisfies long-term usability and robustness requirements. When combined with the CPU, power, and sensor subsystems, the web application ensures that SafeHaven delivers timely, actionable, and reliable information to caregivers. This fully aligns with both functional and non-functional design specifications.

Taken as a whole, the final SafeHaven design can be considered a successful proof-of-concept that meets the core project objective of providing modular, real-time smart home monitoring for seniors. All essential sensing, processing, communication, and web-application functions were implemented and validated experimentally. In particular, the system demonstrated that multiple heterogeneous sensors could be integrated into one coherent architecture that detects hazards, processes data locally, communicates securely with cloud services, and presents meaningful alerts to caregivers through an accessible interface. This level of integration is one of the strongest indicators that the final design met its intended purpose.

Another strength of the final design is its balance between technical performance and practical deployment. The selected sensors are compact and light enough for household installation. The CPU is lightweight and designed for continuous operation, and the web interface organizes information in a form that is understandable for non-technical users. This means the project was not only technically feasible in a lab setting but was also designed with realistic residential use in mind. That practical usability is especially important with the target users being seniors and caregivers. They benefit most from systems that are reliable, unobtrusive, and simple to interpret.

5.2 Use of Advanced Knowledge

The SafeHaven design leveraged advanced knowledge from multiple upper-year Electrical and Computer Engineering courses. Circuit design principles from Electronic Circuits (ECE 140, 240, 340) were applied for sensor interfacing, signal conditioning, and ADC calibration. Concepts from Digital Signal Processing (ECE 207, 222, 313) were implemented to create real-time filters with moving averages and median filtering. This improves sensor reliability and minimizes false positives. Wireless communications expertise from ECE 318 and 414 helped in the selection and optimization of Wi-Fi and MQTT protocols to ensure low-latency, and reliable transmission over 2.4 GHz indoor networks. Computer Networks (ECE 358, 416) guided the development of the layered communication architecture. It also allowed for the integration of OSI principles into a secure TCP/IP stack for IoT telemetry. Software Design and Architectures (ECE 452) principles were used to structure firmware and cloud interactions in a modular, maintainable way. The power circuit design used information from ECE 463. The information on DC-DC converters was especially helpful when building a boost converter with reasonable ripple and consistency. It allowed us to power the required components consistently with minimal noise for the components that used the 5V rail to output an analog signal.

5.3 Creativity, Novelty, Elegance

The CPU subsystem demonstrates creativity and elegance through its proactive, environment-focused design. Unlike conventional systems that react to events, SafeHaven continuously evaluates sensor data using finite-state logic and bitmask-based hazard encoding. This allows for simultaneous detection of multiple hazards without computational overhead. Noise filtering and state evaluation algorithms reduce false alerts. This creates a system that is both accurate and resilient. Another important feature is the integration of multiple sensor types into a cohesive event-driven framework. It uses normalized, percentage-based analog processing to simplify hazard detection and thresholding. Security and reliability are embedded with a TLS-encrypted MQTT communication that contains mutual certificate authentication, NTP-

synchronized timestamps, and PROGMEM storage of credentials. The modular architecture enables seamless sensor addition, firmware updates, and scaling without redesigning the CPU.

The power system demonstrates creativity through its efficient voltage regulation strategy and practical approach to portable energy delivery. Unlike simplistic battery-powered designs, the subsystem integrates a boost converter architecture to step up a variable low-voltage input from AA batteries to a stable 5 V rail required by the ESP32 and sensing components. This enables compatibility with standard consumer batteries while maintaining consistent system operation. The design incorporates efficiency considerations through the selection of a high-performance DC-DC converter. This minimizes power losses and thermal dissipation under varying load conditions. Additionally, the system balances simplicity and scalability by avoiding overly complex battery management systems. It still supports future upgrades such as lithium-based rechargeable solutions. The elegance of the design lies in its ability to provide reliable power using minimal components while maintaining compatibility, portability, and ease of deployment in real-world environments.

The anchors and sensors subsystem demonstrates creativity and elegance through its integration of heterogeneous sensing modalities into a unified, low-complexity detection framework. Rather than relying on a single sensing approach, the system combines temperature, motion, door, water, and pressure sensors to provide comprehensive environmental awareness. A key design innovation lies in the normalization of analog sensor outputs into percentage-based representations. This enables consistent threshold-based hazard evaluation across different sensor types. This abstraction simplifies processing within the CPU while preserving sensitivity to environmental changes. Additionally, the deliberate selection of an infrared break-beam sensor for motion detection reflects a trade-off between detection precision and environmental robustness. This reduces false positives compared to wide-field sensing technologies. The subsystem also incorporates lightweight digital filtering techniques such as moving average and median filters. This enhances signal stability without introducing significant computational overhead. This combination of modular hardware design and efficient signal conditioning results in a scalable and robust sensing architecture.

The web application demonstrates creativity through its integration of real-time data processing with a scalable and secure cloud-based architecture. The system employs a full-stack design consisting of a React frontend, a Node.js backend, and a DynamoDB database. This enables efficient handling of real-time sensor data and user interactions. A key innovative aspect is the transformation of raw sensor telemetry into structured, user-friendly “card-based” representations which abstract complex system states into intuitive visual elements. This design significantly improves usability while preserving access to detailed system information. The backend architecture further enhances system performance through optimized API endpoints and asynchronous data handling. This ensures minimal latency between sensor updates and

user interface display. Security is seamlessly integrated using JSON Web Tokens for authentication and encrypted communication protocols. This ensures data integrity and controlled access. The modular nature of the web application allows for easy expansion of features, such as advanced analytics or predictive alerts, without requiring major architectural changes. This represents a well-designed and extensible system.

The elegance of the design is also visible in the way complex behaviour is represented using relatively simple mechanisms. The use of bitmask-based hazard encoding, percentage-normalized sensor values, and event-driven firmware allows the system to manage multiple hazards efficiently without excessive processing overhead. This is an elegant engineering solution because it reduces complexity while preserving flexibility. This makes it easier to expand the system later with additional sensors or alert conditions. The design is not only functional, but also structurally clean and scalable.

5.4 Quality of Risk Assessment

The CPU subsystem incorporated comprehensive risk mitigation strategies throughout development. Electrical risks such as battery short-circuits, punctures, or overheating were addressed with insulated enclosures and protective casings. Improper sensor alignment, which could produce false negatives or repeated triggers, was mitigated through standardized mounting procedures and extensive pre-testing under various environmental scenarios. Data security risks were managed by transmitting all telemetry over TLS-encrypted MQTT channels. It is combined with secure cloud authentication for role-based access control to protect sensitive sensor information. Hardware risks related to GPIO overcurrent or sensor misconnection were mitigated using inline resistors, voltage regulation, and pre-deployment breadboard testing. To prevent misinterpretation of alerts, the web dashboard displays color-coded hazard states alongside concise notifications. AWS SNS ensures multi-channel alert delivery. These measures collectively ensure that the CPU subsystem operates safely, reliably, and in alignment with the project's objectives. Situations that went wrong, such as intermittent Wi-Fi dropouts or occasional sensor glitches, were addressed with automated reconnection routines and fault-tolerant logic. This demonstrates effective risk identification and mitigation in practice.

The power subsystem incorporates several risk mitigation strategies to address electrical stability, safety, and energy limitations. Key risks include voltage instability, battery depletion, and excessive power dissipation under high load conditions. These risks are mitigated using a regulated boost converter. This ensures a stable 5 V output despite variations in battery voltage during discharge. Thermal and efficiency-related risks are addressed through the selection of a converter with high efficiency and sufficient current capacity. This reduces heat generation and prevents component stress.

The anchors and sensors subsystem addresses risks related to detection accuracy, environmental variability, and sensor reliability. Potential sources of error include signal noise, sensor misalignment, and false triggering due to environmental interference. These risks are mitigated through a combination of hardware and software strategies. Physically, sensors are positioned using standardized placement guidelines to ensure consistent detection coverage. At the signal level, digital filtering techniques, such as moving average and median filtering, are applied to reduce noise and eliminate transient spikes. This significantly improves measurement stability as demonstrated in Figure 4.2. Threshold tuning and adaptive sensitivity adjustments also help reduce false positives while maintaining responsiveness to hazard events. The low false positive and false negative rates reported in Table 4.3 confirm the effectiveness of these mitigation strategies. The modular design of the subsystem further reduces risk by allowing individual sensors to be replaced or recalibrated without affecting overall system operation.

The web application addresses risks associated with data security, system reliability, and user interaction. Security risks, including unauthorized access and data breaches, are mitigated through the implementation of TLS-encrypted communication and token-based authentication using JSON web tokens. These mechanisms ensure that only authenticated users can access system data and that all transmitted information remains secure. Performance-related risks, such as latency increases or system overload under high data throughput, are addressed through scalable cloud infrastructure and optimized API design. This enables efficient handling of real-time data streams. User-level risks, particularly the misinterpretation of alerts, are mitigated through intuitive interface design. Some examples are clear visual indicators with colours and symbols, structured data presentation, and actionable notifications. During testing, minor issues such as delayed updates due to network variability were observed. These were resolved through backend optimization and improved data synchronization. These measures ensure that the web application remains reliable, secure, and effective in delivering critical information to users.

The quality of the project's risk assessment is strengthened by the fact that identified risks were directly connected to both implementation choices and prototype testing results. Risks related to unreliable sensing, communication failures, false alarms, and hardware faults were addressed through specific mitigation strategies that were already described in the report. These include filtering, fault detection, watchdog timers, Wi-Fi reconnection routines, MQTT session recovery, and invalid-value tagging. The prototype results further support the effectiveness of this approach. The system achieved high data acquisition reliability, low latency, and strong detection accuracy. This indicates that many anticipated technical risks were successfully reduced in practice. The low false negative count is important because missed hazardous events represent one of the most serious risks in a senior-monitoring application. The report also reflects a realistic understanding of residual risk by acknowledging limitations. Some examples

are reduced battery runtime under heavy load and partial degradation when some sensor data becomes unavailable. Overall, the risk assessment is credible because it identifies meaningful failure modes, links them to concrete engineering responses, and evaluates their impact using measured prototype performance.

5.5 Student Workload

Tasks assigned per student as of March 11, 2026

Student	Tasks Assigned	Workload %
Fariza	Organized meetings, tasks for all group members, and tracked project progress. Worked with Jingwei and Jordan to develop ESP32 firmware and AWS Lambda code for data processing and communication. Configured AWS infrastructure with MQTT, Lambda, DynamoDB, SNS, and API Gateway to enable secure and scalable system operation. Worked with Ishreet to help integrate AWS backend with the ESP32 microcontroller and web application to ensure reliable end-to-end connectivity and real-time functionality. The workload number is slightly higher due to a desire to take on a team lead/managing position for the project.	20
Ginni	Conceived project idea and defined the system architecture (including	16

	component selection, data flow, and subsystem interactions). Evaluated and ordered sensors and microcontrollers. Established the initial AWS IoT Core environment with Fariza, including MQTT comm, RSA based device authentication, DynamoDB integration & population, and Lambda functions to support product prototyping. Worked closely with hardware team members (Jordan, Jingwei, Jenny) to test and troubleshoot data transmission specifically at the interface between the ESP32 and AWS.	
Ishreet	Developed the full SafeHaven web application, including both frontend and backend. Built the React-based UI (login, dashboard, logs, notifications). Implemented backend APIs using Node.js and Express. Integrated authentication (JWT, bcrypt) and connected the system to AWS DynamoDB for real-time sensor data and alert handling.	16
Jenny	Primarily responsible for the initial development of the ESP32 C code for sensor interfacing, including early firmware implementation for reading and processing sensor	16

	data. Also contributed to sensor testing and troubleshooting during integration to identify and resolve hardware and software issues. Supported acquisition of the sensors and ESP32 CPU components required for prototype development.	
Jingwei	Primarily responsible for the hardware implementation of the sensing subsystem, including circuit design, wiring, sensor interfacing, troubleshooting, and development of sensor-related logic code. Also contributed to the CPU firmware with Fariza and helped implement transmission of sensor data to AWS for backend processing.	16
Jordan	Power circuit design Hardware acquisition Power circuit Initial prototype construction Final prototype construction Circuit verification and testing 3D modelling and printing of housing.	16

List of Figures

Figure 3.1: Architecture of the CPU data handling pipeline. Illustrates how sensor data is structured, processed, and stored within the DynamoDB database for real-time and historical analysis.

Table: SafeHavenSensorDatabaseV2 - Items returned (2,935)

Scan started on March 22, 2026, 21:00:35

☐	deviceId (String)	timestamp (String)	deviceType	payload	unixTime
☐	cpu-01	3/11/2026, 12:41:40 PM	sensors	{ "door": { "...	1773247300
☐	cpu-01	3/11/2026, 12:41:42 PM	sensors	{ "door": { "...	1773247302
☐	cpu-01	3/11/2026, 12:41:44 PM	sensors	{ "door": { "...	1773247304
☐	cpu-01	3/11/2026, 12:41:46 PM	sensors	{ "door": { "...	1773247306
☐	cpu-01	3/11/2026, 12:41:48 PM	sensors	{ "door": { "...	1773247308
☐	cpu-01	3/11/2026, 12:41:50 PM	sensors	{ "door": { "...	1773247310

Figure 3.2: Implementation of the finite-state hazard evaluation model using bitmask encoding. This enables efficient simultaneous detection and representation of multiple hazard conditions.

```
// ----- ALERT EVALUATION -----

const alertMap = {
  "Motion Detected": motion === 1,
  "High Temperature": tempC > DEFAULT_THRESHOLDS.TEMP,
  "Water Leak": water_pct > DEFAULT_THRESHOLDS.WATER,
  "High Pressure": pressure_pct > DEFAULT_THRESHOLDS.PRESSURE,
  "Low Battery": battery_pct < DEFAULT_THRESHOLDS.LOW_BATTERY
};

const alertsActive = Object.keys(alertMap).filter(a => alertMap[a]);
```


Figure 3.3: Event-driven firmware execution flow demonstrating non-blocking task scheduling, interrupt handling, and concurrent processing within the CPU subsystem.

```
// ----- SEND ALERT -----  
  
if (unacknowledgedAlerts.length > 0) {  
  await sendSNS(  
    "🚨 SafeHaven Alert",  
    `Device ${deviceId}: ${unacknowledgedAlerts.join(", ")}`  
  );  
}  
  
// ----- OFFLINE CHECK -----  
  
const offline =  
  door === -1 &&  
  motion === -1 &&  
  tempC === -1 &&  
  water_pct === -1 &&  
  pressure_pct === -1;
```

Figure 3.4: JSON payload structure transmitted by the CPU, including device identification, timestamp synchronization, sensor measurements, and bitmask-encoded hazard states for efficient cloud communication.

```
// ----- FINAL PAYLOAD -----  
  
const payload = {  
  door,  
  motion,  
  tempC,  
  water_pct,  
  pressure_pct,  
  battery_pct,  
  alertsActive,  
  lastAlertTimes,  
  alertAcknowledged,  
  offline,  
  unixTime  
};
```

Figure 3.5: Power system battery life estimation. Illustrates calculated runtime under varying load conditions based on system power consumption.

Peak current draw with all sensors and transmitting data = $460mA$
 Peak current at idle while receiving sensor data = $20mA$
 Boost converter efficiency = 85%
 Updates sent every 2 seconds. Time to send each 1kb package is about $20ms$ worst case.
 Percent of time spent idle = 99%
 Total power of 2S3P AA battery configuration is $\approx 22Wh$
 Board operating voltage = $3.3V$
 Operating Time:

$$\begin{aligned}
 T_{operation} &= \frac{E_{battery}}{P_{operation}} \\
 &= \frac{22Wh}{0.85(0.99 * 0.02 + 0.46 * 0.01)W} \quad (1) \\
 &= 1060.75h \\
 &= 44.198d
 \end{aligned}$$

Figure 3.6: Real-time dashboard displaying live sensor data, hazard states, and system status. It enables continuous monitoring and rapid assessment of environmental conditions.

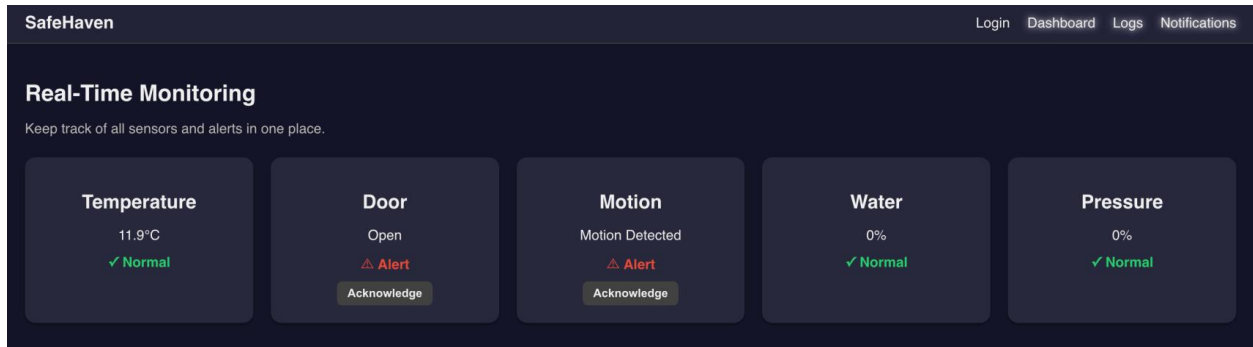
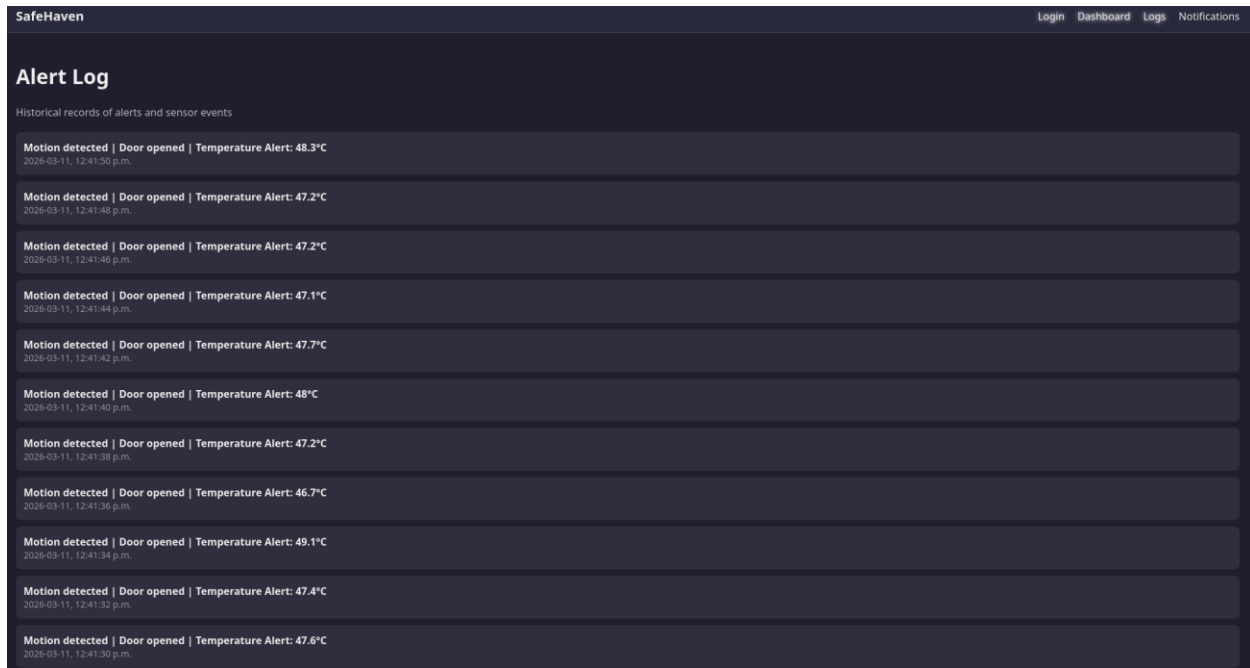
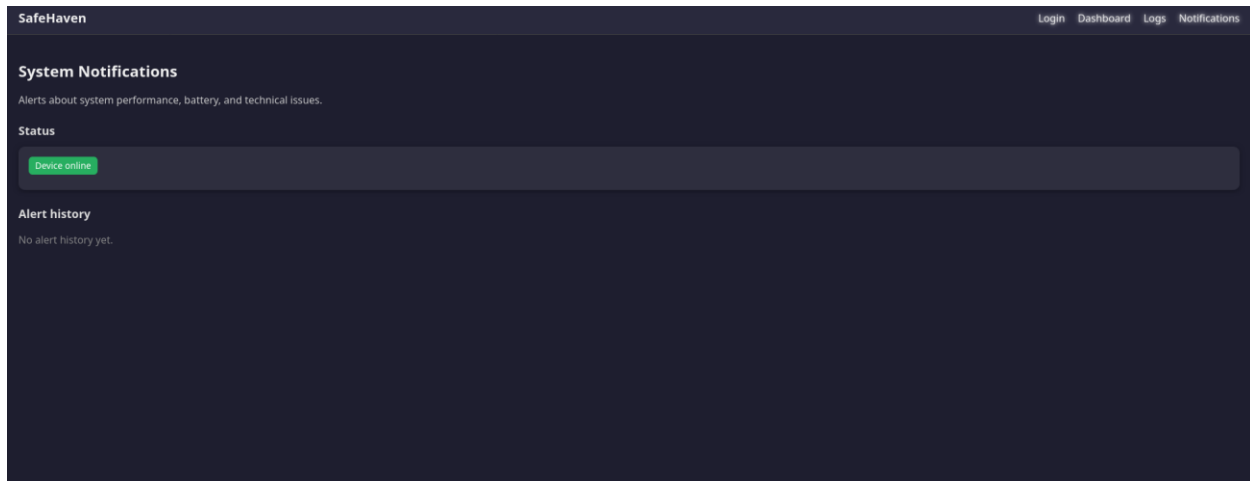


Figure 3.7: Alerts log interface showing a historical record of detected hazard events, such as timestamps, event types, and system responses, for post-event analysis.



Timestamp	Event Details
2026-03-11, 12:41:50 p.m.	Motion detected Door opened Temperature Alert: 48.3°C
2026-03-11, 12:41:48 p.m.	Motion detected Door opened Temperature Alert: 47.2°C
2026-03-11, 12:41:46 p.m.	Motion detected Door opened Temperature Alert: 47.2°C
2026-03-11, 12:41:44 p.m.	Motion detected Door opened Temperature Alert: 47.1°C
2026-03-11, 12:41:42 p.m.	Motion detected Door opened Temperature Alert: 47.7°C
2026-03-11, 12:41:40 p.m.	Motion detected Door opened Temperature Alert: 48°C
2026-03-11, 12:41:38 p.m.	Motion detected Door opened Temperature Alert: 47.2°C
2026-03-11, 12:41:36 p.m.	Motion detected Door opened Temperature Alert: 46.7°C
2026-03-11, 12:41:34 p.m.	Motion detected Door opened Temperature Alert: 49.1°C
2026-03-11, 12:41:32 p.m.	Motion detected Door opened Temperature Alert: 47.4°C
2026-03-11, 12:41:30 p.m.	Motion detected Door opened Temperature Alert: 47.6°C

Figure 3.8: System notifications interface presenting real-time alerts and user notifications. This allows caregivers to acknowledge and respond to critical events promptly.



System Notifications
Alerts about system performance, battery, and technical issues.

Status

Device online

Alert history
No alert history yet.

Figure 4.1: End-to-end system latency convergence plot showing stabilization of processing delay over time. This demonstrates consistent real-time performance.

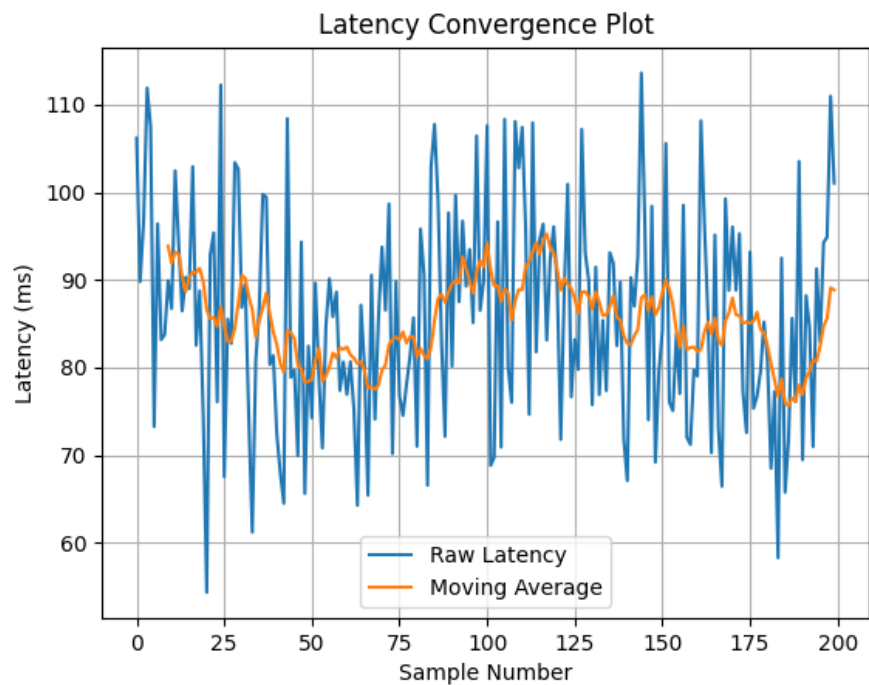
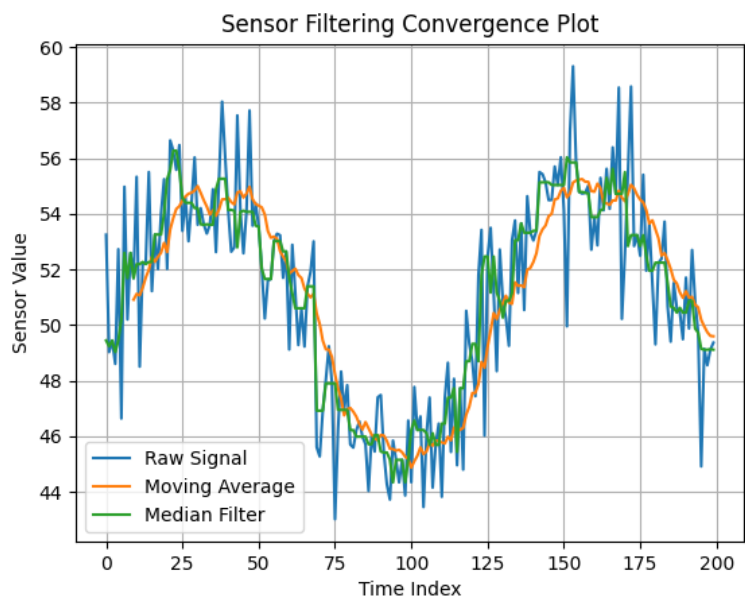


Figure 4.2: Sensor signal filtering comparison illustrating noise reduction between raw and filtered data. Moving average and median filtering techniques are used.



List of Tables

Table 4.1: Performance characteristics of all sensors, including sampling rate, measurement accuracy, noise levels (standard deviation), and response time, under experimental conditions.

Sensors	Type	Sampling Rate	Accuracy	Noise (Std Dev)	Response Time
<i>LM35 Temperature</i>	Digital	5 Hz	$\pm 0.5^{\circ}\text{C}$	0.8°C (filtered)	200 ms
<i>Water Level (Analog)</i>	Digital	5 Hz	$\pm 2\%$	1.2% (filtered)	210 ms
<i>Pressure Mat</i>	Digital	5 Hz	$\pm 1\%$	0.5%	180 ms
<i>Motion Sensor (IR)</i>	Analog	10 Hz	N/A	N/A	150 ms
<i>Door Sensor (Magnetic)</i>	Analog	10 Hz	N/A	N/A	100 ms

Table 4.2: CPU processing latency breakdown. It details time contributions from sensor acquisition, hazard evaluation and filtering, and data transmission.

Task	Average Time	Std Dev
<i>Sensor reading</i>	35 ms	5 ms
<i>Hazard evaluation & filtering</i>	40 ms	6 ms
<i>JSON formation & MQTT publish</i>	10 ms	2 ms
<i>Total latency</i>	85 ms	12 ms

Table 4.3: System reliability metrics summarizing total sensor readings, successful acquisitions, error rates (false positives and false negatives), and overall detection accuracy.

Metric	Result
<i>Total sensor readings attempted</i>	86400
<i>Successful readings</i>	85220
<i>Success rate</i>	98.7%
<i>False positives (Hazard incorrectly triggered)</i>	12 events
<i>False negatives (Missed hazards)</i>	5 events
<i>Overall detection accuracy</i>	98.6%

References

- [1] “Infographic: Canada’s seniors population outlook: Uncharted territory,” CIHI, <https://www.cihi.ca/en/infographic-canadas-seniors-population-outlook-uncharted-territory> (accessed Mar. 25, 2026).
- [2] P. H. A. of Canada, “Government of Canada,” Canada.ca, <https://www.canada.ca/en/public-health/services/publications/diseases-conditions/aging-chronic-diseases-profile-canadian-seniors-report.html> (accessed Mar. 25, 2026).
- [3] “ADT® homepage: Shop adt® Home Alarm Systems: Home Security Systems & Services Provider,” ADT® Homepage | Shop ADT® Home Alarm Systems | Home Security Systems & Services Provider, <https://www.adt.com/> (accessed Mar. 25, 2026).
- [4] “Simplisafe Home Security Systems: Wireless Home Security alarms,” Flag of the United States, <https://simplisafe.com/> (accessed Mar. 25, 2026).
- [5] “Vivint Home Security & Smart Home Systems,” Vivint, <https://www.vivint.com/> (accessed Mar. 25, 2026).
- [6] “Arduino Nano ESP32 with headers [ABX00083] – ESP32-S3, USB-C, wi-fi, bluetooth, HID support, MicroPython compatible for IOT & Embedded Projects,” Amazon.ca: Electronics, https://www.amazon.ca/Arduino-Nano-ESP32-headers-ABX00083/dp/B0C947BHK5/ref=sr_1_5?crd=32YSHUQ2DPJP6&dib=eyJ2ljojMSJ9.GzP-GvhsR81ftjmV7C-hRXHrqc1PH7lpMI0S-AwsGMKGX-wT3lwt5Gf47G9_XJkcvVkQJj_1Gq2DRGPBn-wgfHYl3iF5DR0QRj6b1DGGzpjffJZpQP_edEkknpc5HYavrmnCquob3AxH-Tb60hflKzmVhPL1FU2Z9UZOWIUlu2tEsl8R0NRhqa0gPLRaniXYmTs-ON5oBNZ8AauqvEYMa3_GtOLQtqoix3E-BILctHafuObz58RLOCuH7W-T-RHh20LmZdXy-hxOCipCveky339B2kydKCBklL4p0GSwg8.mgTGtKU7mkqCQ3SMWeyF9vug4KEi_FMNw93m4o1N0ik&dib_tag=se&keywords=Arduino%2BNano%2BESP32&qid=1751583328&sprefix=arduino%2Bnano%2Besp32%2Caps%2C124&sr=8-5 (accessed Mar. 27, 2026).
- [7] “Amazon.ca: Low prices – fast shipping – millions of items,” Amazon.ca: Low Prices – Fast Shipping – Millions of Items, <https://www.amazon.ca/> (accessed Mar. 25, 2026).
- [8] “TEMP-01 - temperature sensor module LM35 based,” SAYAL Electronics, <https://shop.sayal.com/products/1814-cb1> (accessed Mar. 25, 2026).
- [9] “2168 - Adafruit Beam sensor leds,” SAYAL Electronics, https://shop.sayal.com/products/1815-sa3?_pos=13&_sid=caa36012f&_ss=r (accessed Mar. 25, 2026).
- [10] B7-hidden pressure activated mat alarm chime alert/ ..., <https://www.amazon.ca/B7-HIDDEN-PRESSURE-ACTIVATED-INTRUDER-Digiteck/dp/B0045U4MNC> (accessed Mar. 25, 2026).

- [11] Waveshare, <https://www.waveshare.com/esp32-s3-touch-lcd-1.69.htm> (accessed Mar. 26, 2026).
- [12] "Premier online supplier of electronics, leds, Arduino, IOT," PCBoard.ca - LEDs, Electronics, Maker Supplies and More..., <https://www.pcboard.ca/cr2032-battery-holder-with-switch-board> (accessed Mar. 26, 2026).
- [13] Amazon.ca, <https://www.amazon.ca/Solderless-Breadboard-Tie-Point-Circuit-Arduino/dp/B07P1GWN3F?crid=26FGDSVNH2UUF&srefix=breadboard%2Caps%2C112&sr=8-8> (accessed Mar. 26, 2026).
- [14] "ESP32 for IOT: Transmitting data over WIFI using MQTT," Tutorialspoint, https://www.tutorialspoint.com/esp32_for_iot/esp32_for_iot_transmitting_data_over_wifi_using_mqtt.htm (accessed Mar. 26, 2026).
- [15] FreeRTOS.org, "FreeRTOS documentation - freertosTM," FreeRTOS, <https://www.freertos.org/Documentation/00-Overview> (accessed Mar. 27, 2026).
- [16] "LM35 temperature sensor," Components101, <https://components101.com/sensors/lm35-temperature-sensor> (accessed Mar. 27, 2026).
- [17] "MC-38 - Magnetic Switch for Home Alarm System," Components101, <https://components101.com/sensors/mc38-magnetic-switch-sensor-pinout-features-datasheet-working-alternative-application> (accessed Mar. 27, 2026).
- [18] "Ir breakbeam sensors Datasheet by Adafruit Industries LLC," DigiKey, <https://www.digikey.com/en/htmldatasheets/production/2911888/0/0/1/ir-breakbeam-sensors> (accessed Mar. 27, 2026).
- [19] "The standard for IOT messaging," MQTT, <https://mqtt.org/> (accessed Mar. 27, 2026).
- [20] L. E. Staff, "Esp32 basics - analog-to-digital converter (ADC)," Last Minute Engineers, <https://lastminuteengineers.com/esp32-basics-adc/> (accessed Mar. 27, 2026).
- [21] "ESP32 - RTC," ESP32 Tutorial, <https://esp32io.com/tutorials/esp32-rtc> (accessed Mar. 27, 2026).
- [22] "RTC Battery for Raspberry Pi 5, Options For 25mAh / 64mAh / 180mAh Battery Capacity, 2PIN JST connector," Waveshare.com, 2025. <https://www.waveshare.com/rtc-battery.htm?sku=26357> (accessed Mar. 27, 2026).
- [23] "Official Raspberry Pi Pico W Board, built-in WIFI, RP2040 chip, dual-core ARM cortex M0+ (SCO918)," Amazon.ca: Electronics, https://www.amazon.ca/Official-Raspberry-Pi-Built-Dual-core/dp/B0B5H17CMK/ref=sr_1_5?crid (accessed Mar. 27, 2026).

- [24] "Keeyees Development Board WIFI Wlan Wireless Module for ESP8266 NODEMCU ESP-12E CP2102 compatible with Arduino (1PC)," Amazon.ca: Electronics, https://www.amazon.ca/KeeYees-Internet-Development-Wireless-Compatible/dp/B07PR9T5R5/ref=sr_1_5?crd=35Z5Z8MA1MUDB&dib= (accessed Mar. 27, 2026).
- [25] "DHT11 Humidity & Temperature Sensor Technical Data Sheet," Aosong Electronics, Translated Version, Accessed: Mar. 27, 2026. [Online]. Available: <https://www.mouser.com/datasheet/2/758/DHT11-Technical-Data-Sheet-Translated-Version-1143054.pdf>. Accessed: Mar. 27, 2026.
- [26] Aosong Electronics Co., Ltd., "Digital-output relative humidity & temperature sensor/module: DHT22 (DHT22 also named as AM2302)," [Online]. Available: <https://cdn.sparkfun.com/assets/f/7/d/9/c/DHT22.pdf>. Accessed: Mar. 27, 2026.
- [27] "TMP36 Temperature Sensor," Adafruit Learning System. [Online]. Available: <https://learn.adafruit.com/tmp36-temperature-sensor/overview>. Accessed: Mar. 27, 2026.
- [28] "LM35 Temperature Sensor," ElectronicWings. [Online]. Available: <https://www.electronicwings.com/sensors-modules/lm35-temperature-sensor>. Accessed: Mar. 27, 2026.
- [29] "EEMB 3.7V 1100mAh 603449 Rechargeable Lithium Polymer ion Battery Pack with JST Connector.Make Sure Device Polarity Matches with Batteries Before Purchase!!! : Amazon.ca: Health & Personal Care," Amazon.ca. (accessed Mar. 27, 2026). https://www.amazon.ca/dp/B08FD39Y5R?ref=ppx_yo2ov_dt_b_fed_asin_title&th=1 (accessed Mar. 27, 2026).
- [30] ELECFREAKS, "Ultrasonic Ranging Module HC-SR04," [Online]. Available: <https://cdn.sparkfun.com/datasheets/Sensors/Proximity/HCSR04.pdf>. [Accessed: Mar. 27, 2026].
- [31] RCWL, "RCWL-0516 Microwave Radar Sensor Module Datasheet," [Online]. Available: <https://www.makerhero.com/img/files/download/RCWL0516-Datasheet.pdf>. [Accessed: Mar. 27, 2026].
- [32] Adafruit, "Adafruit Mini Magnetic Reed Switch Sensor," [Online]. Available: https://www.adafruit.com/product/3317?srltid=AfmBOopQ_AZh7r0G4yGiPMi5rKak7qkb0nMrfQs1YH1LRL-nuqecwfpE. [Accessed: Mar. 27, 2026].
- [33] SparkFun Electronics, "Reed Switch - Magnetic Switch (Mini)," [Online]. Available: <https://www.sparkfun.com/reed-switch.html>. [Accessed: Mar. 27, 2026].

- [34] ADI Global Distribution, "3W-WS4945 Wireless Magnetic Contact," [Online]. Available: <https://www.en.adiglobaldistribution.ca/Catalog/shopproducts/intrusion/alarm-contacts/magnetic-contacts/3W-WS4945>. [Accessed: Mar. 27, 2026].
- [35] Allelco Electronics, "A Guide to the A3144 Magnetic Hall Effect Sensor," [Online]. Available: <https://www.allelcoelec.com/blog/A-Guide-to-the-A3144-Magnetic-Hall-Effect-Sensor.html>. [Accessed: Mar. 27, 2026].
- [36] "Polling vs. interrupts," RealDigital, <https://www.realdigital.org/doc/52c7417ec4a0c43ae2d328c621b565db#interrupt-controllers> (accessed Mar. 27, 2026).
- [37] PatAltimore, "Publish and subscribe MQTT messages using Azure MQTT broker - azure IOT operations," Publish and Subscribe MQTT Messages Using Azure MQTT Broker - Azure IoT Operations | Microsoft Learn, <https://learn.microsoft.com/en-us/azure/iot-operations/manage-mqtt-broker/overview-broker?tabs=portal> (accessed Mar. 27, 2026).
- [38] Time Series database - amazon timestream - AWS, <https://aws.amazon.com/timestream/> (accessed Mar. 28, 2026).
- [39] Fall detection system with accelerometer and threshold ..., <https://repository.uel.ac.uk/download/a32f7f3eacffce303d8040c71353085af48ba2614f9b57490e47ba73785324b/1727646/Fall%20Detection%20System%20with%20Accelerometer%20and%20Threshold-based%20Algorithm.pdf> (accessed Mar. 28, 2026).
- [40] espressif, "ble taking too much power (IDFGH-769) · Issue #947 · espressif/esp-idf," GitHub, 2025. <https://github.com/espressif/esp-idf/issues/947#issuecomment-500312453> (accessed Mar. 28, 2026).
- [41] "React," React Blog RSS, <https://react.dev/> (accessed Mar. 28, 2026).
- [42] "Low-code programming for event-driven applications," Node, <https://nodered.org/> (accessed Mar. 28, 2026).
- [43] "Vue.js," Vue.js - The Progressive JavaScript Framework | Vue.js, <https://vuejs.org/> (accessed Mar. 29, 2026).
- [44] "Angular," Home • Angular, <https://angular.dev/> (accessed Mar. 29, 2026).
- [45] "1.69inch Touch LCD Module - Waveshare Wiki," Waveshare.com, 2025. https://www.waveshare.com/wiki/1.69inch_Touch_LCD_Module (accessed Mar. 29, 2026).